

UntoldMoney Blueprint

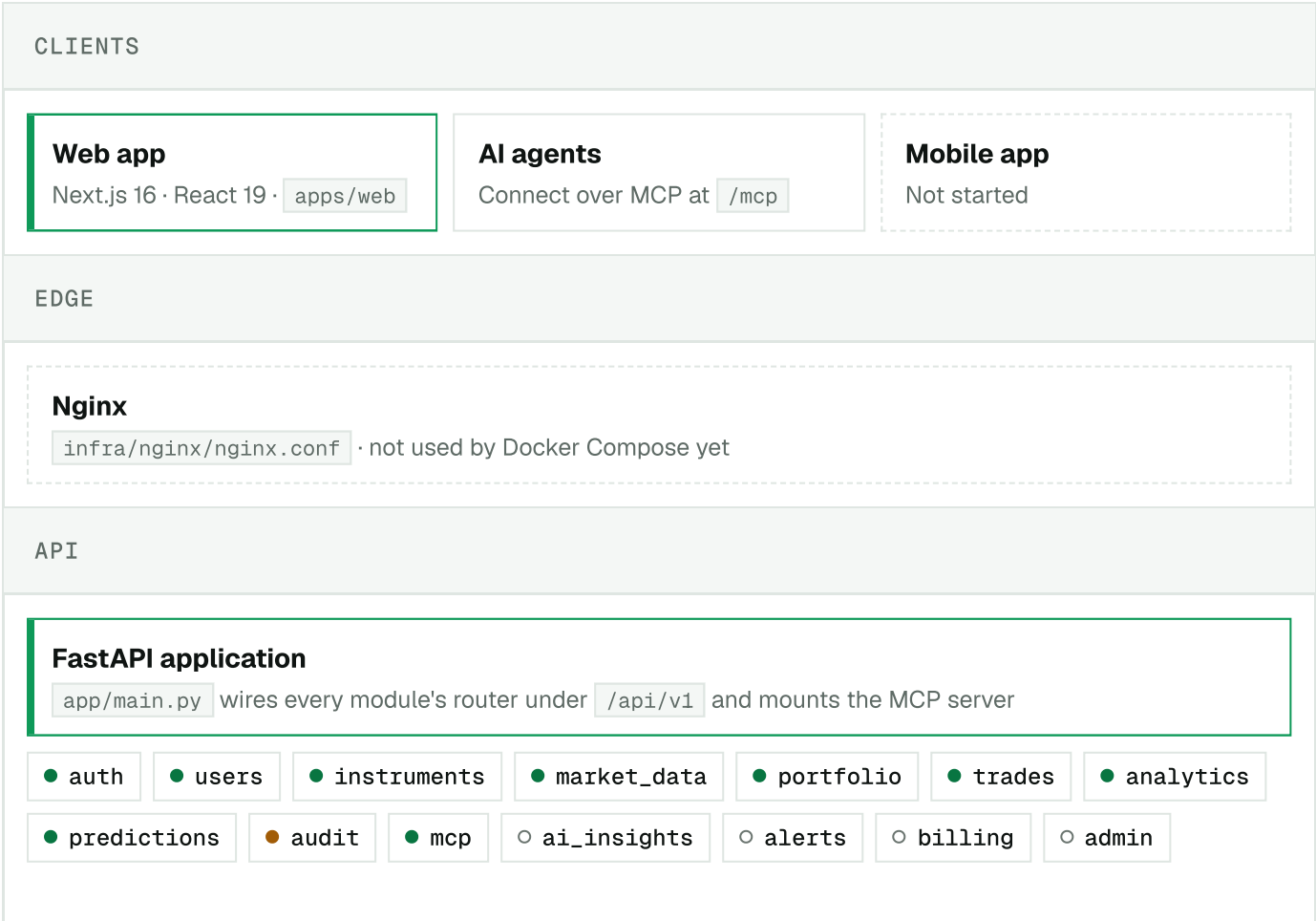
How the platform is built today, what each part does, what stands between it and a production B2C launch, and how to grow it into an AI-native product. Every finding below points to the file it comes from.

Modular monolith One FastAPI app, split into modules	15 tables PostgreSQL via async SQLAlchemy
7 MCP tools Already exposed for AI agents	2 tests Only a health check today

01 · ARCHITECTURE

System map

Read it top to bottom: a request starts at a client and works down to the data. Module status comes from what is actually implemented in `apps/api/app/modules`.



BACKGROUND

Celery worker + beat

Redis broker · `app/workers` · every task currently returns `not_implemented`

DATA

PostgreSQL

Users, portfolios, trades, candles, scores, predictions, audit

Redis

Celery queue · cache (lightly used)

Yahoo Finance

External, via the `yfinance` library

● Built with endpoints ● Partial (models/service only) ○ Empty folder, placeholder

The `services/` folder (analytics-engine, data-ingestion, ml-engine, notification-worker) holds README files only. The design in `docs/h1d.md` keeps everything in one app first and splits services out later. That's the right call at this stage; don't split until a module has a real scaling reason.

02 · DATA FLOW

A request, end to end

Every module follows the same three layers: a **router** handles HTTP, a **service** holds the business rules, and a **repository** talks to the database. Here is what happens when the web app asks for a stock's 3-day direction.

- 1 The browser calls the API through RTK Query, attaching the JWT access token.

`web/store/api/baseApi.ts`

- 2 The router validates input with Pydantic and gets a database session via FastAPI dependency injection.

`predictions/router.py`

- 3 The service checks the `predictions` table for a result less than 24 hours old (cache-aside).

`predictions/service.py`

- 4 On a miss it loads daily candles through the market-data repository.

`market_data/repository.py`

- 5** It trains a gradient-boosting model inside the request, using a time-ordered 80/20 split, then predicts. This blocks the API while it runs.

predictions/ml_engine.py

- 6** The result and its explanation are saved, then returned as JSON.

predictions/repository.py

03 · FEATURES

What each module does

MODULE	WHAT IT DOES	KEY ENDPOINTS	STATUS
auth / users	Register, login, JWT access + refresh tokens, bcrypt passwords, sessions, consent records	/auth/*	● Built
instruments	NSE stock master list, search, exchanges, company info	/instruments , /search	● Built
market_data	Daily OHLCV candles and quotes, Yahoo provider, ingestion runs	/symbol/candles , /quote	● Built
analytics	SMA, EMA, RSI, MACD, ATR, and a 0–100 stock score from trend, momentum, volatility and volume	/symbol/indicators , /score	● Built

MODULE	WHAT IT DOES	KEY ENDPOINTS	STATUS
predictions	ML direction (3-day), price range (5-day), trade-outcome model, with explanations	<code>/predictions/*</code>	● Built
portfolio	Portfolios, holdings, valuation, allocation, 30% concentration warning	<code>/portfolios/*</code>	● Built
trades	Trade journal with strategy, mistake and emotion tags; win rate, profit factor	<code>/trades</code> , <code>/stats</code>	● Built
mcp	Lets AI agents call 7 tools: candles, quote, info, search, compare, indicators, ML prediction	<code>/mcp</code> (SSE)	● Built
audit	Audit log model and service for auth events	none	● Partial
ai_insights · alerts · billing · admin	Planned in the docs; folders contain only <code>__init__.py</code>	none	○ Placeholder

Tech & concepts

Backend

FastAPI + Pydantic v2

Async Python web framework; Pydantic validates every request and response against typed schemas.

SQLAlchemy 2 (async) + Alembic

ORM for PostgreSQL with `asyncpg`; Alembic versions every schema change as a migration file.

Celery + Redis

Runs slow jobs (data sync, model training) outside the request, on a queue.

scikit-learn, pandas, yfinance

Feature engineering and gradient-boosting models over price history from Yahoo Finance.

MCP (FastMCP)

Model Context Protocol: a standard way to expose tools to AI agents.

structlog, python-jose, passlib

Structured JSON logs, JWT signing, bcrypt password hashing.

Frontend

Next.js 16 App Router

File-based routes; `(auth)` and `(dashboard)` are route groups sharing layouts.

React 19 + TypeScript

Server and client components; all dashboard pages are client components.

Tailwind v4 + shadcn on Base UI

Design tokens in `globals.css`; accessible primitives for dialogs, menus, tabs.

Redux Toolkit + RTK Query

Auth state plus API caching per endpoint. Zustand is installed but unused.

react-hook-form + zod

Forms with schema validation shared in `lib/validations.ts`.

Recharts, Phosphor icons

Charts and the duotone icon set.

Infra & tooling

Docker Compose

Postgres, Redis, API, web, Celery worker and beat for local development.

Nginx, Terraform

Config stubs in `infra/`; not wired into deployment yet.

pytest, ruff, mypy, ESLint

Installed and configured; test coverage is minimal.

Docs suite

HLD, LLD, API contract, schema, security plan, compliance boundary and roadmap in `docs/`.

Modular monolith

One deployable app with strict module boundaries, so a module can become a service later without a rewrite.

Router → service → repository

HTTP concerns, business rules and SQL live in separate layers, which keeps each one testable on its own.

Dependency injection

FastAPI's `Depends` hands each request its DB session and current user.

Access + refresh tokens

Short-lived access JWT for calls; a longer-lived refresh token gets new ones.

Cache-aside

Predictions are read from the table first and recomputed only when older than 24 hours.

Time-ordered validation

Train on older data, test on newer data, so the model never sees the future (no look-ahead bias).

Feature engineering

Turning raw candles into model inputs: returns, moving-average gaps, RSI, MACD, volatility, volume ratios.

Compliance boundary

`docs/compliance-boundary.md`: analytics and education only, no advice, no execution.

What stands between this and production

Verified against the code. Fix the critical items before any real user signs up.

CRITICAL

The MCP endpoint has no authentication

Anyone who can reach `/mcp` can call every tool. `app/main.py · mount("/mcp")`

Fix: Require a scoped API key or OAuth token, rate-limit it per client, and log every tool call to audit.

CRITICAL

Tokens are stored in localStorage

Any injected script can read and steal them. `web/lib/auth.ts`

Fix: Move to httpOnly, Secure, SameSite cookies set by the API, with refresh-token rotation and reuse detection.

CRITICAL

Dashboard routes are not protected

The middleware lets every request through. `web/middleware.ts`

Fix: Check the session in `proxy.ts` (Next 16's replacement for middleware) and redirect signed-out users; keep authorization on the API too.

CRITICAL

"Buy" and "Strong Buy" labels conflict with your own compliance doc

The score emits `STRONG_BUY` / `BUY` (`analytics/service.py:359`), while `docs/compliance-boundary.md` rules out buy/sell signals. In India, buy/sell ratings for retail users can fall under SEBI's Research Analyst rules.

Fix: Rename to descriptive bands (for example "Strong uptrend score"), or obtain registration. Get a securities lawyer's view before a B2C launch.

HIGH

Models train inside API requests

CPU-heavy scikit-learn runs inside an async handler and blocks other users' requests. `predictions/service.py`

Fix: Train in Celery on a schedule, save versioned models to a registry, and only load and predict at request time.

HIGH

Market data comes from an unofficial source

`yfinance` scrapes Yahoo Finance; its terms don't allow commercial redistribution and it can break without notice.

Fix: License NSE data from an authorized vendor, keep the provider interface you already have, and add data-quality checks and corporate-action adjustments.

HIGH

Almost no tests

One test file with two health checks. `app/tests/`

Fix: Add service-level unit tests, API tests against a test database, and frontend component tests; run them in CI on every pull request.

HIGH

Background jobs are empty

Every Celery task returns `not_implemented` and the beat schedule is empty, so prices never refresh on their own. `app/workers/`

Fix: Implement ingestion, scoring and alert jobs with retries, idempotency keys and run records.

MEDIUM

No rate limiting

A `RateLimitException` class exists but nothing enforces limits. `core/exceptions.py`

Fix: Add Redis-backed limits per user and IP, stricter on login and on AI endpoints.

MEDIUM

Migration failures are hidden

Docker Compose runs `alembic upgrade head 2>/dev/null || echo ...`, so a broken migration still starts the API.

Fix: Run migrations as a separate deploy step that fails loudly.

MEDIUM

The dashboard shows mock data

Stat cards, recent trades and top movers are hardcoded. `(dashboard)/dashboard/page.tsx`

Fix: Back each widget with a real endpoint and add loading, empty and error states.

MEDIUM

No error tracking or metrics

Logs are structured, but there's no tracing, metrics or alerting.

06 **Fix:** Add OpenTelemetry traces, Prometheus metrics, Sentry for errors, and uptime checks.

Making it AI-native, the robust way

You already have the hardest part: deterministic tools (indicators, scores, predictions) exposed over MCP. Agents should reason over those tools and never invent numbers. Build one shared AI platform first, then each agent on top of it.

LLM gateway

One internal module (fill the empty `ai_insights`) that every agent calls. It handles provider choice, model routing (a strong model for multi-step reasoning, a small fast one for tagging), prompt caching, retries, timeouts, per-user token quotas and cost tracking.

Tool layer

Reuse the 7 MCP tools, scoped to the signed-in user, and add portfolio and journal tools. Tools return numbers; the model only explains them.

Agent runtime

Long runs execute in Celery as recorded *agent runs*: status, each step, tool calls, tokens and cost, stored in Postgres and streamed to the UI over SSE.

Memory & retrieval

Conversation history and user preferences in Postgres; `pgvector` for searching filings, news and the user's own journal notes.

Guardrails

A compliance check on every output (no advice wording, required disclaimers), a grounding check that each number appears in a tool result, PII redaction, and every AI response written to the audit log.

Evaluation

Golden question sets per agent, automated grading in CI, and a review queue for flagged answers. Ship an agent only when its scores hold.

Agent 1 · build first

Stock research copilot

Chat about any stock. Pulls candles, indicators, score and prediction, then explains them in plain language with the numbers cited.

`get_stock_candles` · `get_technical_indicators` · `get_ml_prediction`

Agent 2

Trade journal coach

Reads tagged trades, finds repeated mistakes and emotional patterns, and proposes process rules, never trades.

`trades` · `stats` · `tag performance`

Agent 3

Weekly portfolio review

Scheduled run: concentration, sector exposure, drawdown and what changed, delivered as a short report.

`portfolio summary` · `allocation` · `scores`

Agent 4

Natural-language alerts

"Tell me when Reliance RSI drops below 30" becomes a structured alert rule the worker evaluates.

`alerts (to build)` · `indicators`

Agent 5

Filings & news digest

Summarizes corporate announcements and results for the user's holdings, with links to sources.

`document retrieval` · `instruments`

Agent 6

Natural-language screener

"NSE stocks with RSI under 30 and rising volume" becomes a safe, parameterized query over your data.

`search` · `indicators` · `scores`

Development roadmap

Ordered by dependency: each phase relies on the one before. Move on only when its exit check passes.

Phase 0 Harden
• Cookie-based auth with refresh rotation; route protection • Authenticate and rate-limit MCP and APIs • Rename buy/sell labels; legal review • CI: lint, type-check, tests, build on every PR • Sentry, OpenTelemetry, secrets in a vault
EXIT CHECK Security review passes; CI blocks merges; 70% coverage on services.
Phase 1 Real data
• Licensed NSE data vendor behind the provider interface • Scheduled ingestion with retries and run records • Corporate-action adjustments; data-quality checks • TimescaleDB hypertables for candles
EXIT CHECK Prices refresh daily without manual steps; data gaps raise alerts.
Phase 2 ML platform
• Offline training jobs; MLflow model registry • Backtests and calibration shown to users • Drift monitoring; automatic rollback to the last good model
EXIT CHECK No training in requests; prediction p95 under 300 ms.

Phase 3

AI layer

- LLM gateway, guardrails, eval harness
- Research copilot, then journal coach, then portfolio review
- Agent run records and streaming UI

EXIT CHECK

Each agent passes its eval set; cost per active user is tracked.

Phase 4

B2C product

- Plans and subscriptions (Razorpay for India); feature quotas
- Alerts and notifications: email, push, WhatsApp
- Onboarding; installable web app, then mobile
- DPDP Act 2023: consent, data export and deletion

EXIT CHECK

Paid sign-up to first insight works end to end without support.

Phase 5

Scale

- Load tests; autoscaling API and workers
- Backups with point-in-time recovery; restore drills
- Runbooks, on-call, status page

EXIT CHECK

Defined uptime target met for 30 days; a restore drill succeeds.

08 · DIFFERENTIATORS

Features worth adding

Chosen to fit the compliance boundary: they explain, organize and coach, and none of them tells users what to buy.

CAS statement import

Upload the NSDL/CDSL consolidated account statement PDF and the portfolio fills itself. No manual holdings entry.

India-specific; removes the biggest onboarding chore.

Read-only broker sync

Connect a broker account to pull holdings and executed trades into the journal automatically.

The journal becomes complete without typing.

Trade replay

Replay any journaled trade on the chart, showing what the indicators and score looked like at entry and exit.

Turns the journal into a learning loop.

Behavior fingerprint

Detects patterns like trading more after losses or exiting winners early, using timestamps and emotion tags.

Feeds the journal coach agent.

Why the score changed

A daily diff: "63, up from 58, mostly from momentum," with the inputs that moved.

Makes the score explainable over time.

Portfolio stress test

"What if IT stocks fall 10%?" applied to the user's real holdings and sectors.

Risk education without predictions.

Capital-gains view

Estimated short- and long-term gains per holding, by Indian tax rules, clearly marked as an estimate.

A frequent need at year end.

Suggested next step: start Phase 0. The four critical findings are each a few days of work, and everything else, especially AI features that touch user data, depends on them.

